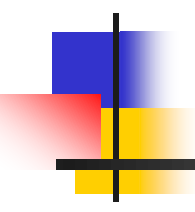


Assembly Language Fundamentals



Introduction

- 
- Assembly Language programs are translated into machine language by an assembler, so they must be written to conform to assemblers specification (We shall use [MASM 6.11](#))
 - Assembly Language code is generally not case sensitive but one should stick to lower or upper case to differentiate code and rest of the text.

Assembly Language Syntax

- Programs consists of **statements** one per line (can be extended in next line by inserting backslash "\ " character)
- Each statement is either an **instruction** , which the assembler translates to machine code or an **assembler directive**, which directs the assembler to perform some specific task.

Assembly Language Syntax

- Both Instructions and directives may have up to four fields:

[name] [operation] [operand(s)] [;comments]

- At least one blank or tab character must separate these fields.
- Fields do not have to align in a particular column but they must appear in above order.

Assembly Language Syntax (The *Statement*)

Example of Instruction:

[name] **[operation]** **[operand(s)]** **[/comments]**

Start: **mov** **cx, 10h** **/initialize**

counter

In above example

- name = start:
- operation = mov
- operands = cx and 10h
- comments = /initialize counter

Assembly Language Syntax (The *Statement*)

Example of Directive:

[name] [operation] [operand(s)] [;comments]

```
main    proc
```

In above example

- name = main
- operation = proc
- This particular directive creates a procedure called **main**



Assembly Language Syntax (The *name* field)

- The assembler translates **names** into memory addresses.
Names are used for **labels**, **procedure** and **variables**.
- A name assigned in program data segment is called **variable**.
- A name assigned in program code segment is called **label**.
- Name can be of 1~247 characters long (MASM)
- Embedded blanks are not allowed.
- No distinction between uppercase and lowercase letters.
- A **assembler reserved word** cannot be used as name.

Assembly Language Syntax (The name field)

- Valid “name” characters:

- A,B,CZ, a,b,c.....z, 0,1,29, ?,_,@,\$,

- Example of Legal Names

- counter1
- @character
- sum_of_integers
- \$amount
- ?done

- Example of Illegal Names

- one word
- 2xyz
- you&me
- wx.yz

Assembly Language Syntax (The Operation field)



➤ For an **instruction**, the operation field consists of a symbolic operation code (**opcode** or **Mnemonic**). The assembler translates a symbolic opcode into machine language opcode. Opcode symbols often describe the operation function e.g.,

mov, add, sub etc

➤ In an **assembler directive**, the operation field contains a pseudo-operation code (**pseudo-op**). Pseudo-ops are not translated into machine code, rather they tell the assembler to do something e.g., **proc** pseudo-op is used to create a procedure.

Assembly Language Syntax (The Operand (s) field)

For an instruction, the operand field specifies the data that are to be acted upon by the operation. An instruction may have zero, one or two operands e.g.,

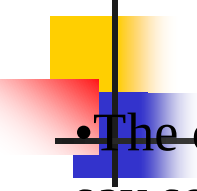
hlt ;Halts the processor

inc ax ;add 1 to contents of register ax

add ax,bx ;add the contents of register ax and bx and
;store in register ax

First operand is called destination operand and second operand is called source operand.

Assembly Language Syntax (The Comments field)

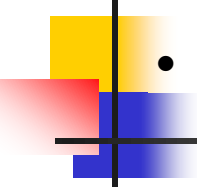


- The comment field of statement is used by the programmer to say something about what the statement does.

- Comments are started with semicolon (;), assembler ignores anything typed after semicolon.

- It is a good programming practice to write comments with each statement.

Numeric Literals (Constants)

- 
- A numeric literal is any combination of digits, plus:
 - optional decimal point, exponent, sign
 - examples of decimal literals:

5234

5.5

-5.5

26.0E+05

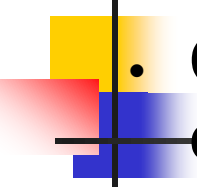
+35d

Numeric Literals (Constants)

- Use a radix symbol (suffix) to select binary, octal, decimal, or hexadecimal

6A15h	; hexadecimal
0BAF1h	; leading zero required
32q	; octal
1011b	; binary
35d	; decimal (default)

Symbolic Constant

- 
- Created by assigning a name to some constant expression.
 - Defined using the = operator

```
COUNT = 25
```

```
ROWS = 10
```

```
tablePos = ROWS * 5
```

Constant Expression

- Combination of numeric literals, operators, and defined symbolic constants
 - must be evaluated at assembly time

• examples:

```
4 * 20
```

```
-3 * 4 / 6
```

```
ROWS - 3 ; (ROWS is a constant)
```

Character or String Constants

- A constant may also represent a string of characters enclosed in either single or double quotation marks. (Embedded quotes are also permitted.

- examples:

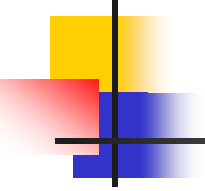
```
"ABC"
```

```
'x'
```

```
'Display "what is your name?" '
```

```
"4096"
```


The Hello World Program



```
title Hello World Program                (hello.asm)

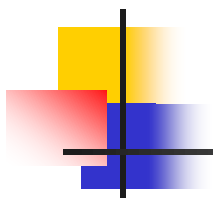
; This program displays "Hello, world!"

.model small
.stack 100h
.data
message db "Hello, world!", '$'

.code
main proc
    mov ax,@data
    mov ds,ax
    mov ah,9
    mov dx,offset message
    int 21h

    mov ah,4Ch
    int 21h
main endp
end main
```

Sample Hello Program



```
title Hello World Program          (hello.asm)

; This program displays "Hello, world!"

.model small

.stack 100h
```

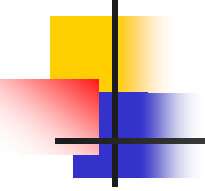
program title (comment)

comment line

memory model

set the stack size

Sample Hello Program



```
.data
message db "Hello, world!", '$'

.code
main proc
    mov ax, @data
    mov ds, ax
    mov ah, 9
    mov dx, offset message
    int 21h

    mov ah, 4Ch
    int 21h
main endp
end main
```

starts the data segment

starts the code segment

sets DS to the actual base value of the data segment

Sets DX to the offset value of "message" in data segment

calls DOS display function 9

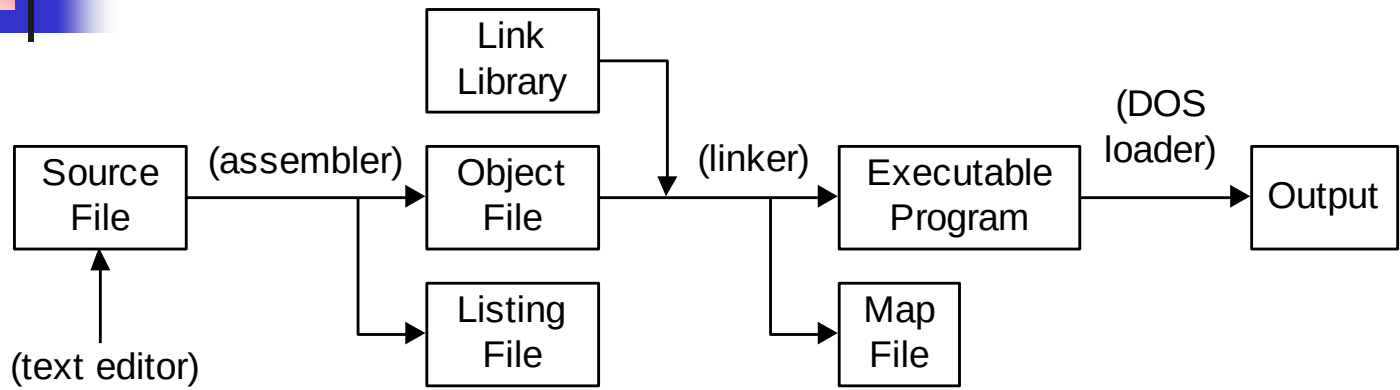
halts program

Common Assembler Directives



Directive	Description
end	End of program assembly
endp	End of procedure
proc	Begin procedure
title	Title of the listing file
.code	Mark the beginning of the code segment
.data	Mark the beginning of the data segment
.model	Specify the program's memory model
.stack	Set the size of the stack segment

Assemble-Link-Execute Cycle



Files Created by the Assembler and Linker



Filename	Description	Step When Created
hello.asm	Source program	Edit
hello.obj	Object program	Assembly
hello.lst	Listing file	Assembly
hello.exe	Executable program	Link
hello.map	Map file	Link

Data Allocation Directives



Mnemonic	Description	Bytes	Attribute
DB	Define byte	1	Byte
DW	Define word	2	Word
DD	Define doubleword	4	Doubleword

Define Byte (DB)

- Create data having a **byte** attribute
 - one or more bytes may be defined together
 - can mix different data representations
 - assigns a starting value of allocated data

```
.data
char1      db    'A'
signed1    db    -128
combo      db    'A', -10, 30h, 40q, 1101b
list       db    10,20,30,40
aString    db    "Hello there"
count      db    ?                ; uninitialized
```


Define Byte (DB)



.data

alist db 'abcd'

The name **alist** identifies only the offset of the first character “a”

e.g., if “a” is stores at offset	0000
then “b”	0001
“c”	0002
“d”	0003

DUP Operator

- Use DUP to create an array
 - each element is assigned the same value

```
.data
arrayB db 30 dup(10h)
array2 db 10 DUP(?)          ; uninitialized
array3 db 5 Dup(0)
```

DUP operator can also be nested:

```
aTable db 4 dup( 3 dup(0) , 2 dup('x' ) )
```

This example creates storage containing

000xx000xx000xx000xx

Define Word (DW)

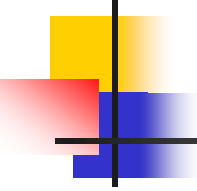
- Create data having a **word** attribute
 - can mix different data representations
 - can use integer expressions
 - can contain 16-bit offset of another variable called **pointer**.

```
.data
list      dw  1234h,5000h,65535, -24
signed1    dw  35 * 4
aString    dw  "AB"
ptr        dw  list ;ptr points to list
```

Define Doubleword (DD)

- Create one or more 32-bit integers
-
- doubleword** attribute

```
.data  
largeVal    dd 12345678h  
array       dd 100 dup(?)  
smallVals   dd -1,-2,-3,-4,-5
```



Symbolic Constants

To make assembly language code easy to understand, it is often desirable to use a symbolic name for a constant value.

Equal-Sign Directive

Known as **redefinable** equate, In contrast to **db** and **dw** directives equal sign directive allocates no storage, instead all occurrence of names are replaced by expression.

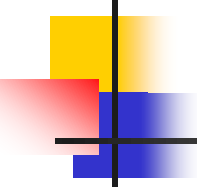
```
prod      = 10 * 5          ; Evaluates an expression
maxInt     = 7FFFh          ; Maximum 16-bit signed value
minInt     = 8000h          ; Minimum 16-bit signed value
maxUInt    = 0FFFFh        ; Maximum 16-bit unsigned value
string     = 'XY'
count      = 500
endvalue   = count + 1      ; Can use a predefined symbol
```

.386

```
maxLong    = 7FFFFFFFFh     ; Maximum 32-bit signed value
minLong    = 80000000h      ; Minimum 32-bit signed value
maxULong   = 0FFFFFFFFh     ; Maximum 32-bit unsigned value
```

You can redefine any number of time in your program.

Using the Equal-Sign Directive



Statement	Assembled As
count = 5	
mov al,count	mov al,5
mov dl,count	mov dl,5
count = 10	
mov cx,count	mov cx,10
mov dx,count	mov dx,10
count = 2000	
mov ax,count	mov ax,2000

EQU Directive

- A symbol defined with EQU cannot be redefined later in the program.

Syntax: name EQU constant

e.g., In case of ab EQU 10h directive, machine code of following instruction will be same.

mov ah,10h and mov ah, ab

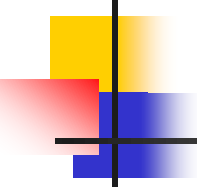
Symbol can also be string of characters.e.g., in case of following directive:

q EQU "type your name"

message db "type your name" or

message db q will be same for assembler.

Data Transfer Instructions

- 
-
- MOV Instruction
 - Operands with Displacements
 - XCHG Instruction

MOV Instruction



.data

count db 10

total dw 4126h

bigval dd 12345678h

.code

mov al,bl ;8-bit register to register

mov bl, count ;8-bit memory to register

mov count, 26 ;8-bit immediate to memory

mov bl, 1 ;8-bit immediate to register

mov dx,cx ;16-bit register to register

mov bx, 8FE2h ;16-bit immediate to register

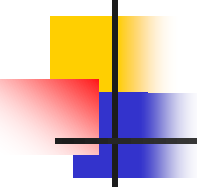
mov total 1000h ;16-bit immediate to memory

mov edx,ebx ;32-bit register to register

mov edx, bigval ;32-bit memory to register

Both operands cannot be memory locations.

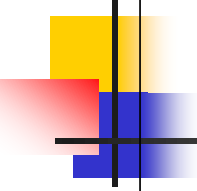
XCHG Instruction



Examples

xchg	ax, bx	;exchange two 16-bit registers
xchg	ah, al	;exchange two 8-bit registers
xchg	var1, bx	;exchange two 16-bit memory ;operand with BX
xchg	eax, ebx	;exchange two 32-bit register

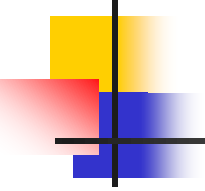
Program to exchange Two Variables

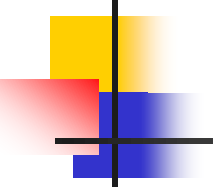


```
title Exchange Two Variables      (Exchange.asm)
.model small
.stack 100h
.data
value1 db 0Ah
value2 db 14h
.code
main proc
    mov ax,@data      ; initialize DS register
    mov ds,ax
    mov al,value1      ; load the AL register
    xchg value2,al      ; exchange AL and value2
    mov value1,al      ; store AL back into value1

    mov ax,4C00h      ; exit program
    int 21h
main endp
end main
```

Arithmetic Instructions

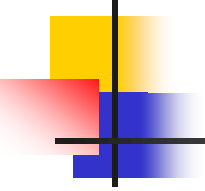
- 
- INC and DEC
 - ADD
 - SUB
 - Flags affected by ADD and SUB



INC and DEC

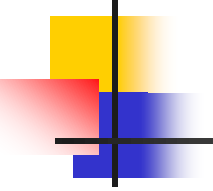
- INC
 - adds 1 to destination operand
- DEC
 - subtracts 1 from destination operand
- for both instructions:
 - operand can be either a register or variable
 - Carry flag not affected

INC and DEC Examples



```
.data
membyte      db      25
memword      dw      36
doubleVal    dd      12340000h

.code
inc al
dec bx
inc eax
inc membyte
inc memword
dec doubleVal
```



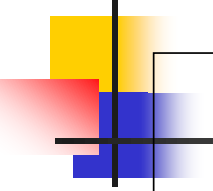
ADD Instruction

ADD destination, source

Adds source operand to destination operand

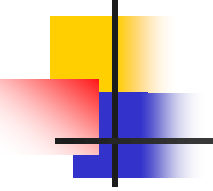
- Affects the Carry, Overflow, Sign and Zero flags
- source operand can be register, immediate value, or memory
- destination operand can be register or memory
- only one operand can be a memory operand

ADD Instruction Examples



```
.data
membyte    db 25
memword     dw 36
doubleVal   dd 12340000h
.code
add  al,5
add  bx,ax
add  eax,edx
add  membyte,al
add  memword,bx
add  doubleVal,edx
```

SUB Instruction

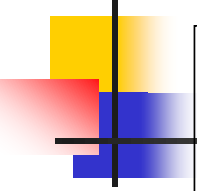


SUB destination, source

Subtracts source operand from destination operand

- Affects the Carry, Overflow, Sign and Zero flags
- source operand can be register, immediate value, or memory
- destination operand can be register or memory
- only one operand can be a memory operand

SUB Instruction Examples



```
.data
membyte db 25
memword dw 36
doubleVal dd 12340000h
.code
sub al,5
sub bx,ax
sub eax,edx
sub membyte,al
sub memword,bx
sub doubleVal,edx
```

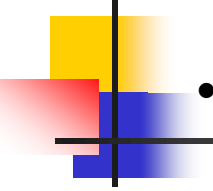
Flags Affected by ADD and SUB



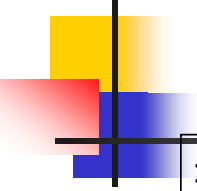
After the instruction has executed,...

- If the destination is zero, the **Zero** flag is set
- If the destination is negative, the **Sign** flag is set
- If there was a carry out of the highest bit of the destination, the **Carry** flag is set

Signed Overflow

- 
- Signed overflow occurs when adding two signed integers, if and only if...
 - both operands are positive, or both operands are negative and the sign of the sum is opposite to the sign of the values being added

```
mov  al,+127
add  al,+1    ; AL = 80h (-128), Overflow
```



Signed Overflow Examples

```
mov  al,+127
add  al,+1          ; AL = 80h (-128), Overflow

mov  dx,-32768      ; DX = 8000h (-32768)
add  dx,-1          ; DX = 8001h, Overflow

mov  dx,-32768      ; DX = 8000h (-32768)
sub  dx,1           ; DX = 8001h, Overflow
```

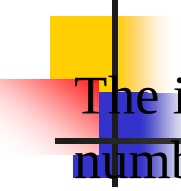
Basic Operand Types



Three Basic types of operands:

- 1- Immediate:** a constant value
- 2- Register :** one of the CPU registers
- 3- Memory:** reference to a location in memory

Immediate Operands



The immediate operand is a constant expression, such as a number, character constant, arithmetic expression or symbolic constant.

mov	bx,10	;number
mov	eax,12345678h	;number
mov	dl, "X"	;character constant
mov	ax, (40*50)	;arithmetic expression
mov	cx, count	;symbolic constant, count = 20

Register Operands



A Register operand can be any register (except IP).

Instructions having only registers as operand execute most efficiently as no memory access is required.

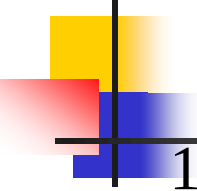
Some examples with register operands:

```
mov    eax,ebx
```

```
mov    cl,20h
```

```
mov    si, offset message
```

Memory Operands



1- Direct Operands

2- Direct Offset Operands

3- Register Indirect Operands.

Memory Operands

1- Direct Operands:

A direct operand refers to contents of memory at a location identified by a name in the data segment. e.g.,

.data

count db 20

word dw 1000h,2000h

dword dd 12345678h

.code

mov al,count

mov bx,word

mov ecx,dword

Memory Operands

2- Direct Offset Operands

This method can be used to use memory locations as operand that have no names e.g.,

.data

```
array db 0ah,0bh,0ch,0dh
```

.code

```
mov al,array ; AL = 0Ah
```

```
mov bl,array+1 ;BL = 0Bh
```

```
mov cl,array+2 ;CL = 0Ch
```

```
mov dl,array+3 ;DL = 0Dh
```

Memory Operands

3- Register Indirect

Contents of the CPU registers are used to point a memory location.

```
mov    ax, [si]
```

```
mov    [bx], ax
```